

Phản dịch hợp ngữ và tối ưu hằng số thời gian

Ghi chú theo slide

Zhou Yisu

2006

Nguồn gốc: Disassembly for time-constant optimization

IOI China candidate team papers / 2006 / Zhou Yisu / Zhou Yisu.ppt

1 Phạm vi

Bộ slide đi kèm bài viết về tối ưu hằng số thời gian bằng cách quan sát mã hợp ngữ do trình biên dịch sinh ra. Ghi chú này không thay thế phần bài viết đầy đủ; nó gom lại các điểm nên nhớ khi đọc slide.

2 Vì sao xét hằng số

Trong thi lập trình, hai chương trình có cùng độ phức tạp tiệm cận vẫn có thể khác nhau rất xa về thời gian chạy. Khi giới hạn dữ liệu sát với khả năng của thuật toán, chi phí của phép chia, truy cập bộ nhớ, gọi hàm, truyền tham số, hoặc kiểm tra nhánh đều có thể quyết định kết quả.

3 Cách nhìn qua hợp ngữ

Phản dịch hợp ngữ giúp người viết chương trình thấy một câu lệnh C++ được biến thành bao nhiêu thao tác máy. Một phép gán mảng đơn giản thường khác một phép truy cập qua con trỏ nhiều tầng; một vòng lặp có điều kiện phức tạp có thể sinh thêm nhiều nhánh; phép chia nguyên thường đắt hơn cộng, trừ và dịch bit.

4 Cải tiến thực dụng

Các hướng tối ưu trong bài viết gồm:

- giảm phép toán đắt trong vòng lặp nóng;
- đưa biểu thức không đổi ra ngoài vòng lặp;
- chọn kiểu dữ liệu và cách truyền tham số phù hợp;
- thay đổi bố cục dữ liệu để truy cập bộ nhớ tuần tự hơn;
- kiểm tra mã sinh ra sau khi tối ưu để tránh sửa nhầm vào chỗ không ảnh hưởng đáng kể.

5 Giới hạn

Tối ưu hằng số chỉ có ý nghĩa sau khi thuật toán lớn đã đúng hướng. Nếu độ phức tạp tiệm cận sai, việc viết sát máy chỉ che được một phần rất nhỏ. Ngược lại, khi mô hình đã đúng và nút thắt nằm trong cài đặt, đọc mã hợp ngữ giúp ta đưa ra quyết định dựa trên bằng chứng thay vì cảm giác.